```
    for (j = 0; j < NUM_FRAMES; j++)
        if (txd[j].ctrl & DESC_EN) { all_done = 0; break; }
    if (all_done) { tx_ok = 1; break; }
}
```

Summary

What	Why
`WR(GRETH_STATUS, STS_ERRORS)` causes infinite loop	Clears error bits but GRETH stays disabled — TXIRQ/RXIRQ never fires
TX loop exits fine	No TX AHB error — TX DMA path works
RX loop hangs	RX AHB error → GRETH RX disabled → RXIRQ never comes
Root cause	Either rxbuf address unreachable, or wrong RX descriptor length field
Fix 1	Remove length from RX descriptor ctrl (GRETH fills it in)
Fix 2	Break immediately on AHB error bits — don't wait for IRQ

The `printf("RX AHB error")` line will confirm this is what's happening when you run it.

Using GRETH_STATUS CAUSED INFINITE LOOP. MY MISTAKE..IT DIDN'T COMPLETE THE TX FOR LOOP BUT RATHER THE BUILDING OF DESCRIPTOR LOOP. ALSO SINCE WE HAD PUT THE TX_OK=1 INSIDED THE IF(STS & STS_TXIRQ) IT WAS ERRORLY PRINTING ALL TX FRAMES TRANSMITTTED. ACTUALLY IN THE FOR LOOP THE TX ONLY WENT INSIDE THAT IF LOOP ONLY ONCE..(SAME IS THE CASE FOR RX) ... I TRIED THAT IF(STS & STS_TXAHB) AND IF(STS & STS_TXERR) AND

Show more

This is very clear now. The AHB error is the core problem. Let me analyze exactly what is happening.

Deep analysis of AHB error root cause >

Give me

Ran a command, read a file >

Here is a summary of every change and why:

Fix 1 — Separate TX and RX phases (the main fix)

Previously both `CTRL_TXEN` and `CTRL_RXEN` were enabled together. This means GRETH's TX DMA and RX DMA were both trying to use the AHB bus at the same time — TX to read frame data from `txbuf`, RX to write received data to `rxbuf`. On frame 2 onwards they collided causing an AHB error.

The fix runs TX completely first with only `CTRL_TXEN` set. All 4 loopback frames accumulate in the PHY/MAC receive FIFO during this time. Then RX is enabled separately with only `CTRL_RXEN` to drain all 4 frames. No AHB contention.

Fix 2 — Re-assert only the relevant enable bit

When `STS_TXIRQ` fires only `CTRL_TXEN` is re-written. When `STS_RXIRQ` fires only `CTRL_RXEN` is re-written. This stops each engine from accidentally triggering the other.

Fix 3 — Break immediately on AHB errors with a print

Instead of clearing `STS_ERRORS` and waiting (which hangs because GRETH disables itself after an AHB error and never sets TXIRQ/RXIRQ), the code now checks each error bit individually, prints which error occurred and at which iteration, and breaks immediately. The descriptor status prints after each phase will show exactly which frame failed.

Fix 4 — `tx_ok` / `rx_ok` only set when ALL descriptors are done

Previously `tx_ok=1` was set inside `if(STS_TXIRQ)` which fires once per frame — so it was incorrectly set to 1 after just the first frame. Now it is only set when the inner loop confirms all `DESC_EN` bits are cleared.

Fix 5 — Test 1 does not set `CTRL_TXEN` / `CTRL_RXEN`

Without descriptors GRETH immediately self-clears those bits. Test 1 only tests register access so only `CTRL_FULLD` is written.

Fix 6 — RX descriptor has no length field pre-filled

From the PDF the LENGTH field in an RX descriptor is written by GRETH after reception to show how many bytes were received. We should not pre-fill it. `rxd[i].ctrl = DESC_EN` (plus `DESC_WRAP` on last). GRETH fills in the length itself.

Greth test fixed
C
Download

If this doesn't work what else should i try

Good thinking to plan ahead. Here is every possible remaining issue ranked by likelihood:

Level 1 — Most Likely (check these first)

1a — AHB error still occurs even with separated TX/RX

If you still see `TX AHB error` or `RX AHB error` in the transcript after the fix, the problem is not bus contention but something more fundamental.

What to check:

c

```
// Add these prints right after greth_reset() in Test 5:
printf("txd  @ 0x%08X\n", (unsigned int)txd);
printf("rxd  @ 0x%08X\n", (unsigned int)rxd);
printf("txbuf@ 0x%08X\n", (unsigned int)txbuf[0]);
printf("rxbuf@ 0x%08X\n", (unsigned int)rxbuf[0]);
printf("NRD  = %d\n", (RD(GRETH_STATUS) >> 24) & 0xF);
```

Addresses must be in SRAM range (0x40000000–0x4FFFFFFF). If they are elsewhere GRETH AHB master cannot reach them.

1b — GRETH AHB master locked by EDCL

Even with EDCL operation disabled, EDCL shares the same AHB master interface as GRETH DMA. If EDCL is still processing something internally it can hold the AHB bus and cause your DMA to get an error response.

Fix:

c

```
// In greth_reset(), after delay(5000), explicitly disable EDCL:
// Read current CTRL value and OR in the ED bit (bit14)
unsigned int ctrl_val = RD(GRETH_CTRL);
WR(GRETH_CTRL, ctrl_val | (1 << 14));  // set ED=1 to disable EDCL
delay(1000);
```

1c — Cache coherency — LEON3 data cache holds stale data

LEON3 has a data cache. When you write to `txbuf` the CPU writes to cache, not necessarily to main memory. When GRETH DMA tries to read `txbuf` from AHB, it reads main memory — which may still have old data or zeros. Same issue on RX: GRETH writes to `rxbuf` in memory but when your code reads `rxbuf` it reads stale cache data.

Fix — flush cache before TX, invalidate before RX check:

c

```
// Add this function:
static void cache_flush(void)
{
    /* LEON3 cache flush via ASI 0x10 (instruction cache flush)
```

```
     * and ASI 0x11 (data cache flush) */
    asm volatile("sta %%g0, [%%g0] 0x11" : : : "memory");
    asm volatile("nop; nop; nop; nop");
}

// Call before enabling GRETH in Tests 4 and 5:
cache_flush();
WR(GRETH_TXDESC, (unsigned int)txd);
WR(GRETH_RXDESC, (unsigned int)rxd);
WR(GRETH_CTRL, CTRL_TXEN | CTRL_FULLD | CTRL_PROM);
```

Level 2 — Moderately Likely

2a — Descriptor table size mismatch (NRD)

From PDF: GRETH's descriptor table size is set by the `NRD` field in STATUS register bits[27:24]. If NRD says the table has 128 entries but GRETH cannot read entries 4–127 because they are not mapped, you get an AHB error when GRETH tries to scan past your 4 descriptors even though WRAP is set on descriptor 3.

Check NRD value:

c

```
unsigned int nrd_val = (RD(GRETH_STATUS) >> 24) & 0xF;
printf("NRD = %d (table = %d entries)\n", nrd_val, 128 << nrd_val);
// If NRD=0 → 128 entries → table occupies 1024 bytes
// Your txd aligned(1024) should cover this ✓
// But confirm: sizeof(txd) = 4*8 = 32 bytes
// Remaining 992 bytes of the 1024 block are uninitialized
// GRETH scans those too if WRAP doesn't stop it
```

Fix if NRD is large:

c

```
// Declare larger descriptor arrays to cover full NRD range
// If NRD=1 → 256 entries → need 256*8 = 2048 bytes
static desc_t txd[128] __attribute__((aligned(1024)));
static desc_t rxd[128] __attribute__((aligned(1024)));
// memset the whole thing to 0 so GRETH reads valid (disabled) descriptors
memset(txd, 0, sizeof(txd));  // 128*8 = 1024 bytes all zero = all disabled
memset(rxd, 0, sizeof(rxd));
```

2b — WRAP bit not stopping the ring correctly

If GRETH ignores the WRAP bit (a rare simulation model bug) it will keep scanning past descriptor 3 into garbage memory.

Check on waveform: After TX frame 3 is sent, does GRETH's internal descriptor pointer wrap back to 0 or keep incrementing?

Alternative fix — use automatic 1KB boundary wrap instead of WRAP bit:

```
c

// Instead of setting DESC_WRAP on last descriptor,
// declare exactly 128 descriptors (fills the whole 1KB)
// GRETH will auto-wrap at the 1KB boundary per PDF:
// "The pointer automatically wraps to zero when the
//  1 kB boundary of the descriptor table is reached"
static desc_t txd[128] __attribute__((aligned(1024)));
// Set DESC_EN only on the 4 you need, rest stay 0 (disabled)
// GRETH processes 0,1,2,3 then hits disabled desc 4 and stops
// No WRAP bit needed
```

2c — PHY RX FIFO overflow during TX phase

During the TX-only phase, 4 frames are transmitted and loop back. The PHY simulation model stores them in its internal RX FIFO. If the FIFO is small (1–2 frames in the simulation model) frames 3 and 4 may be dropped before RX is enabled.

Check `phy.vhd`**:** Look for FIFO depth or buffer size generics.

Fix if FIFO is small:

```
c

// Transmit one frame at a time and receive it before sending next
for (i = 0; i < NUM_FRAMES; i++) {
    // TX frame i
    WR(GRETH_CTRL, CTRL_TXEN | CTRL_FULLD | CTRL_PROM);
    // wait for TXIRQ...
    // RX frame i immediately
    WR(GRETH_CTRL, CTRL_RXEN | CTRL_FULLD | CTRL_PROM);
    // wait for RXIRQ...
}
```

Level 3 — Less Likely but Worth Knowing

3a — greth.vhd simulation model has a bug in multi-frame mode

Some older GRLIB GRETH models have known issues with descriptor ring handling in simulation. If you have access to the GRETH VHDL source, check if `grethm` or `greth` is used and what version.

3b — AHB bus arbitration timeout

If LEON3's AHB arbiter gives a timeout response to GRETH's DMA request (because another master holds the bus too long) GRETH sees it as an AHB error. Check the AHB arbiter configuration in `leon3mp.vhd`.

3c — Address alignment issue on descriptor 1+

Descriptor 0 at `txd` base address is always word-aligned. But if the compiler packs `desc_t` differently than expected, `txd[1]` might not be at `base+8`. Verify:

```c
printf("sizeof(desc_t) = %d\n", sizeof(desc_t));  // must be 8
printf("txd[0] @ 0x%08X\n", (unsigned int)&txd[0]);
printf("txd[1] @ 0x%08X\n", (unsigned int)&txd[1]);
// gap must be exactly 8 bytes
```

Decision Tree — What to Do First

```
Still seeing TX AHB error?
    ↓ YES
    → Check buffer addresses (Level 1a)
    → Try cache flush (Level 1c)
    → Check NRD and expand descriptor array (Level 2a)

TX passes but RX AHB error?
    ↓ YES
    → PHY FIFO overflow — do frame-by-frame TX/RX (Level 2c)
    → Cache coherency on rxbuf reads (Level 1c)

TX and RX pass but data mismatch?
    ↓ YES
    → Cache coherency — rxbuf has stale data (Level 1c)
    → Verify rxbuf addresses are correct per frame
```

```
Everything passes but frames 2/3 still owned by HW?
    ↓ YES
    → NRD mismatch — expand descriptor array (Level 2a)
    → PHY FIFO overflow — frame-by-frame approach (Level 2c)
    → Try removing WRAP bit and rely on 1KB auto-wrap (Level 2b)
```

Try them in the order shown — Level 1 first, then 2, then 3. The transcript prints already in the code will tell you exactly which branch of the tree you are in.

What should be idle responce for all the above tests

Here is the exact idle state for every signal during each test:

Test 1 — Register Access (APB only)

Signal	Idle state during Test 1
TXD[3:0]	0000 (flat, no data)
TX_EN	0 (flat low)
TX_ER	0 (flat low)
RXD[3:0]	0000 (flat, no data)
RX_DV	0 (flat low)
RX_ER	0 (flat low)
MDC	0 (flat, no clock)
MDIO_O	1 (pulled high — idle bus state)
MDIO_I	Z/1 (pulled high by testbench emdio<=H)
TX_CLK	toggling (PHY generates this always)
RX_CLK	toggling (PHY generates this always)
APB PSEL	pulses high briefly for each register write/read
APB PADDR	changes to 0x00,0x08,0x0C per register accessed
APB PWDATA	shows values being written (0x200, 0x8, etc)
APB PRDATA	shows values read back

Test 2 — PHY MDIO Read (MDIO only)

Signal	Idle state / activity during Test 2
TXD[3:0]	0000 (flat, no data — no Ethernet TX)

```
TX_EN           0     (flat low throughout)
RXD[3:0]        0000  (flat)
RX_DV           0     (flat low)
MDC             ACTIVE — 64 pulses per read × 3 reads = 192 pulses total
MDIO_O          ACTIVE — preamble + header bits driven by GRETH
MDIO_I          ACTIVE — PHY drives data bits during turnaround+data phase
                Between transactions: pulled high (idle)
APB PSEL        pulses for each GRETH_MDIO register read/write
```

Between the 3 MDIO transactions:

```
MDC             flat 0  (idle between transactions)
MDIO_O          1       (idle high)
MDIO_I          1/Z     (pulled high by testbench)
```

Test 3 — PHY Loopback Write (MDIO write + read)

```
Signal          Idle state / activity during Test 3
──────────────────────────────────────────────────────
TXD[3:0]        0000  (flat — no Ethernet TX yet)
TX_EN           0     (flat low)
RXD[3:0]        0000  (flat)
RX_DV           0     (flat low)
MDC             ACTIVE — 64 pulses for write + 64 pulses for readback
MDIO_O          ACTIVE — write transaction drives all 64 bits
                         (preamble + header + 16-bit data 0x6100)
MDIO_I          Z     during write (PHY listens, does not drive)
                ACTIVE during readback turnaround + data phase
```

After Test 3 completes and before Test 4 starts:

```
MDC             flat 0
MDIO_O          1 (idle high)
MDIO_I          1/Z
TXD/RXD         flat 0
TX_EN/RX_DV     flat 0
```

Test 4 — Single Frame Loopback

```
Phase           Signal          State
──────────────────────────────────────────────────────────

TX phase:
                TX_EN           0 → pulses HIGH for 64 bytes → 0
                TXD[3:0]        0 → nibbles of frame data → 0
                                First nibbles: F,F,F,F,F,F (dst MAC)
                                Then: 0,2,0,0,0,0 (src MAC)
                                Then payload bytes
                TX_ER           0 (flat — no error)
                RX_DV           0 (flat during TX — loopback not instant)
                RXD[3:0]        0 (flat during TX)

Loopback delay: (PHY processes frame internally)
                All signals flat briefly

RX phase:
                RX_DV           0 → pulses HIGH for 64 bytes → 0
                RXD[3:0]        0 → same nibbles as TXD → 0
                TX_EN           0 (flat — TX done)
                TXD[3:0]        0 (flat)

Idle between TX and RX:
                TX_EN           0
                TXD[3:0]        0000
                RX_DV           0
                RXD[3:0]        0000
                MDC             0
                MDIO_O          1
```

Exact TX_EN pulse width at 100Mb MII:

```
64 bytes × 8 bits = 512 bits
MII sends 4 bits per clock (nibbles)
512 / 4 = 128 clock cycles of TX_EN high
At 25MHz (100Mb MII clock) = 128 × 40ns = 5.12 microseconds
```

Test 5 — Multi-Frame Stress Loopback

```
TX phase (4 frames back to back):

Frame 0:  TX_EN ─┐        ┌── gap ──┐
                 └────────┘         │
```

```
Frame 1:                                  ┐_┐       ┌─
                                            └_______┘

Frame 2:  (same pattern)
Frame 3:  (same pattern, last frame)

Between frames (inter-frame gap):
          TX_EN    = 0
          TXD[3:0] = 0000
          RX_DV    = 0    (loopback frames not yet received)
          RXD[3:0] = 0000

RX phase (after all TX done, CTRL_RXEN enabled):

Frame 0 comes back:  RX_DV ─┐       ┌──
                             └_______┘

Frame 1 comes back:  (same pattern)
Frame 2 comes back:  (same pattern)
Frame 3 comes back:  (same pattern)

During entire TX phase:   RX_DV = 0, RXD = 0000
During entire RX phase:   TX_EN = 0, TXD = 0000
MDC/MDIO:                 flat 0 / high throughout Tests 4 and 5
```

Complete Idle Signal Table

Signal	Idle value	Active during
`TXD[3:0]`	`0000`	Test 4 TX, Test 5 TX phase
`TX_EN`	`0`	Test 4 TX, Test 5 TX phase
`TX_ER`	`0`	Never (no errors expected)
`RXD[3:0]`	`0000`	Test 4 RX, Test 5 RX phase
`RX_DV`	`0`	Test 4 RX, Test 5 RX phase
`RX_ER`	`0`	Never (no errors expected)
`MDC`	`0`	Tests 2 and 3 only
`MDIO_O`	`1` (high)	Tests 2 and 3 only
`MDIO_I`	`Z/1` (pulled)	Tests 2 and 3 data phase

Signal	Idle value	Active during
`TX_CLK`	always toggling	PHY always generates
`RX_CLK`	always toggling	PHY always generates
`APB PSEL`	`0`	Every register access

Key Things to Verify on Waveform

- ✅ `MDC is flat (0) during Tests 1, 4, 5 — no stray MDIO activity`
- ✅ `TX_EN is flat (0) during Tests 1, 2, 3 — no premature TX`
- ✅ `RX_DV is flat (0) during TX phase of Tests 4, 5`
- ✅ `TX_EN goes flat (0) before RX_DV rises — TX finishes before RX starts`
- ✅ `RXD nibbles match TXD nibbles exactly — loopback data integrity`
- ✅ `TX_ER stays 0 throughout — no transmission errors`
- ✅ `RX_ER stays 0 throughout — no reception errors`
- ✅ `4 separate TX_EN pulses in Test 5 TX phase`
- ✅ `4 separate RX_DV pulses in Test 5 RX phase`
- ✅ `Gap between Test 5 TX_EN falling and RX_DV rising — PHY processing time`

If any of these are wrong it tells you exactly where the problem is:

What you see	Problem
MDC toggling during Test 4/5	EDCL FSM still running MDIO
TX_EN never rises	Descriptor not set up, TE self-cleared
RX_DV never rises	PHY loopback not active
RX_DV rises during TX phase	PHY loopback responding too fast — OK if data matches
TX_ER pulses	Collision or underrun — check AHB speed
RX_ER pulses	CRC error or alignment error on received frame
TXD has X values	Uninitialized txbuf or AHB can't read it
RXD has X values	PHY not driving — loopback not set

Give me the document of all the changes we have made from the start

Viewed a file, ran a command >

give me the document and the fixed code file

Ran a command, read a file >

Here is a quick summary of what is in each file:

Change Log Document covers 10 sections and 16 bugs total:

- Section 2 — CTRL register fixes (CTRL_PROM wrong bit, CTRL_100MB not valid in MII, TE/RE without descriptors)
- Section 3 — MDIO register fixes (BUSY wrong bit, READ/WRITE opcodes swapped, PHYSHIFT/REGSHIFT wrong, LINKFAIL check wrong, missing pre-poll, Test 3 wrong busy-poll)
- Section 4 — Descriptor fixes (RX length field, memset before Test 5, descriptor pointer order)
- Section 5 — STATUS register fixes (AHB error infinite loop, stale flags)
- Section 6 — TX/RX flow fixes (tx_ok flag wrong, TE/RE not re-asserted, AHB bus contention from simultaneous TX+RX, wrong engine re-enabled)
- Section 7 — PHY loopback fix (auto-neg must be disabled)
- Section 8 — Reset flow fix (no complex PHY init needed)
- Section 9 — Complete summary table of all 16 changes
- Section 10 — Expected VCS transcript when all tests pass

Fixed code has all current fixes applied including the separated TX and RX phases to eliminate AHB bus contention.

Greth test change log
Document · DOCX
Download

Greth test fixed
C
Download

Download all